Windows

From a fresh Windows 10/11 PC to a working token: build the virtual HID driver, install it, start the bridge. The top-level `README.md` explains the hardware and wiring (Part A steps 1-2) and the firmware; this page replaces its Linux-specific steps. A PDF version of this page is in README-windows.pdf.

All scripts are plain `.cmd` files (no PowerShell scripts, no MSBuild needed). Run them from PowerShell or Command Prompt; from the repository root the paths are written as `desktop\windows\...`.

Status

Part	Status
Serial link (COM ports), token detection, `fpctl.py`	implemented, host tests in CI
Virtual FIDO device (`pico_vhf.sys` + `pico-vhid.exe`)	built and installed on the author's PC (Windows 10/11 x64, WDK 10.0.26100), the bridge creates the device `1D50:619B` and finds the token
`.cmd` scripts (`build`, `install`, `uninstall`, `check-token`, `setup-token`, `start-bridge`)	used on that PC
Browser registration (Edge, Chrome)	not written up here yet

Files

File	Purpose
`vhid\build.cmd`	builds `pico-vhid.exe` and `pico-install.exe` (MinGW) and the driver package (Visual Studio + WDK)
`vhid\install.cmd` / `vhid\uninstall.cmd`	install / remove the driver (administrator)
`setup-token.cmd`	interactive: enrol / test / delete fingers, initialise keys, FIDO reset, start the bridge
`check-token.cmd`	read-only status; exit code 0 = ready
`start-bridge.cmd`	runs `..\common\ctaphid_serial_bridge.py`, which starts `vhid\pico-vhid.exe` (administrator)
`common.cmd`	helpers shared by the scripts
`vhid\`	driver, helper, installer sources, vhid/README.md
`virtual-hid.md`	the options for a virtual FIDO device on Windows and the decision

1. Tools to install (once)

Tool	Used for	Notes
Python 3 with the `py` launcher	bridge and `fpctl.py` (no extra packages)	python.org, tick "Add python.exe to PATH"
winlibs MinGW-w64, **x86_64**	`build.cmd tools` (gcc)	winlibs.com, unzip, add its `bin` folder to PATH. Check: `gcc -dumpmachine` prints `x86_64-w64-mingw32`
Visual Studio 2022 with "Desktop development with C++"	`cl.exe`, `link.exe` for the driver	Community is fine
Windows 10/11 SDK, then the **WDK with the same version number**	KMDF/VHF headers and libs, `stampinf`, `Inf2Cat`, `signtool`	install the SDK first, then the WDK; the Visual Studio extension is not needed
libfido2 tools (`fido2-token.exe`)	`setup-token.cmd` options 5 and 6, `check-token.cmd`	optional for fingerprints only; put the folder on PATH
ESP-IDF v5.5	building and flashing the firmware	Espressif's "ESP-IDF Tools Installer"; flash through the board's **UART** port

Check the WDK: this file must exist (the version folder is the one of your SDK):

```
dir "%ProgramFiles(x86)%\Windows Kits\10\Include\*\shared\vhf.h"
```

2. Build

From a normal prompt in `desktop\windows\vhid`:

```
.\build.cmd tools     pico-vhid.exe and pico-install.exe (MinGW)
.\build.cmd driver    pico_vhf.sys, .inf, .cat, .cer (cl, link, stampinf, Inf2Cat, signtool)
.\build.cmd           both
```

The driver package lands in `vhid\driver\x64\Release\pico_vhf\`. On the first run the script creates the test certificate `CN=PicoVhfTestCert` in your personal certificate store (one PowerShell command inside `build.cmd`). Optional environment variables: `WDKVER` (SDK/WDK version folder), `KMDF_VER` (default 1.15), `WERROR=1`, `WDKContentRoot` (if the WDK is not under `Program Files (x86)\Windows Kits\10`). `cl` prints two C4324 warnings from a KMDF header; they are harmless.

3. Install the driver (administrator)

Open PowerShell or Command Prompt **as administrator**, then:

```
desktop\windows\vhid\install.cmd
```

It asks before each system change:

1. **Test-signing mode** (`bcdedit /set testsigning on`). Needs Secure Boot off and a **reboot**; run `install.cmd` again afterwards. The check looks for the English text `testsigning Yes`.
2. **Trust the test certificate** that signed the package (machine Trusted Root and Trusted Publishers).
3. **Create** `Root\PicoVhf` **and install the driver.**

Verify (administrator):

```
pnputil /enum-devices /problem
```

It must not list "Pico FIDO2 virtual HID source (serial bridge)". Device Manager shows that device under Human Interface Devices. The FIDO HID device itself appears only while the bridge runs.

4. Set up the token (once)

Plug a USB **data** cable into the board's **native USB** port (it shows as "USB Serial Device (COMn)"). Then:

```
desktop\windows\check-token.cmd         status: token, sensor, fingers enrolled
desktop\windows\setup-token.cmd
    menu: 1 enrol, 2 test, 3/4 delete, 5 init keys, 6 new identity, 7 start bridge
```

Enrol at least one finger (three touches); without a finger the token refuses everything. `fpctl.py` and the bridge use the same COM port, so close the bridge before counting or changing fingers (`fpctl.py` also works while the bridge runs, through its control port, but `check-token.cmd` cannot count fingers then). Menu option 5 sends the first FIDO message (a brand-new token creates its keys then); option 6 erases accounts and the PIN, not the fingerprints, and only works within 10 seconds after power-up (the script asks you to replug).

5. Start the bridge (every time)

1. Plug the token into the native USB port, wait about 3 seconds.
2. Administrator prompt: `desktop\windows\start-bridge.cmd` (`-v` prints every packet). **Leave the window open.**
3. Use the browser (Edge, Chrome, Firefox reach the key through Windows' WebAuthn service, nothing to set up), for example https://www.token2.com/tools/fido2-demo. When the bridge window prints `>>> TOUCH THE FINGERPRINT SENSOR NOW <<<`, touch the sensor with an enrolled finger.
4. Stop with Ctrl+C (answer Y to "Terminate batch job").

The output should contain `token on COMn` and `virtual FIDO device 1d50:619b created`, and **no** line saying the virtual HID helper exited. Unplugging is fine: the bridge reconnects.

Uninstall (administrator, bridge stopped)

```
desktop\windows\vhid\uninstall.cmd
```

Removes the device and the driver package, then asks whether to delete the `PicoVhfTestCert` certificate and turn test-signing off (needs a reboot). Fingerprints and token keys are not touched.

What needs administrator rights

Step	Admin?
install / uninstall	yes
`start-bridge.cmd` (opening the driver)	yes
`setup-token.cmd` options 5, 6, 7	yes
fingerprint options 1-4, `check-token.cmd`	no
using the token in a browser	no
`fido2-token -L` / `-I` on the device	yes

Checking the serial side without the driver

From `desktop\common`:

```
py -3 ctaphid_serial_bridge.py --list
    the token's COM port, marked "-> token"
py -3 fpctl.py status
    "token on COMn", then "sensor OK, N fingerprint(s) enrolled"
py -3 ctaphid_serial_bridge.py --no-hid -v
    bridge without a virtual device; unplug/replug: "serial link lost", then "token on COMn"
```

Troubleshooting

What you see	Likely cause	What to do
`gcc not found` / "targets i686"	wrong or missing MinGW	install the x86_64 winlibs build, `gcc -dumpmachine`
"no version folder there has shared\vhf.h"	WDK missing, or not the same version as the SDK	install the WDK with the SDK's version; or set `WDKVER` / `WDKContentRoot`
`cl.exe not found`	Visual Studio C++ workload missing	install it, or use the "x64 Native Tools Command Prompt"
`vhfkm.lib not found`	WDK libraries missing	reinstall the WDK
`install.cmd` asks for test-signing every time	non-English Windows (the check is English only)	answer `n`, run `bcdedit /set testsigning on` yourself, reboot
device in Device Manager with a yellow mark, `pnputil /enum-devices /problem` lists it	driver did not start	code 52: test-signing not active (Secure Boot, reboot). Code 31: the driver failed in `EvtDeviceAdd`; note the status and see `vhid\README.md`
`the pico_vhf driver is not installed` (from `pico-vhid`)	driver missing or not started	`install.cmd`, then check `pnputil /enum-devices /problem`
`cannot find ctaphid_serial_bridge.py and fpctl.py`	`desktop\common` missing	keep the repository layout (`desktop\common` next to `desktop\windows`), or `set PICO_BRIDGE_DIR=...`
`no token found`	wrong USB port, charge-only cable, another program has the port, bridge running	native USB port, data cable, close serial monitors and the bridge
`the bridge is already running`	a bridge in another window	Ctrl+C there
`needs an administrator Command Prompt`	normal prompt	open as administrator
`fido2-token -L` lists only `windows://hello`	not an administrator prompt	run it as administrator

What you see	Likely cause	What to do
reset says `NOT_ALLOWED`	the 10-second window had passed	choose option 6 again and replug at once
browser waits forever, no `host->token` lines with `-v`	Windows does not see the virtual device	Device Manager, Human Interface Devices: is a FIDO device present while the bridge runs?

Known limits

- The driver is test-signed: it needs test-signing mode (lowers Windows' driver-signature protection; turn it off with `uninstall.cmd` when you stop using the token). A release would need an EV certificate and Microsoft attestation signing.
- `install.cmd` / `uninstall.cmd` read the English `bcdedit` output.
- `uninstall.cmd` removes only the `PicoVhfTestCert` certificate; one from another build is removed in `certmgr.msc`.
- A driver package built elsewhere (CI, MSBuild) has no `pico_vhf.cer`; `install.cmd` then extracts it from the `.cat` with one PowerShell command.